Exercise - Understanding clock tree

This exercise is still being updated for the 2026FS class. You can look at it but don't expect it to be in its final version until this message is displayed.

About the style

We will use a number of different styles to identify different types of actions as shown below:

Student Task

Parts of the text that have a gray background, like the current paragraph, indicate steps required to complete the exercise.

Actions that require you to select a specific menu will be shown like the following: `menu → sub-menu → sub-sub-menu`

Whenever there is an option or a tab that can be found in the current view/menu we will use a `BUTTON` to indicate such an option.

Throughout the exercise you will be asked to enter certain commands using the command-line.

```
sh> command to be entered on the Linux command line
```

Note: *there are many reasons for using a command-line. Some functionality can not not be accessed through GUI commands, and in some cases, using the command-line will be much faster. Most importantly, things you enter on the command-line can be converted into a script and executed repeatedly.*

Overview

The clock net is a special net for every synchronous digital design: you already noticed from the placement&timing exercise that special timing constraints hold for the clocks of the system. In this exercise, we will explain how the clock is distributed to all the design flip-flops. We will motivate why the clock net requires a special distribution network in the chip and consider important parameters of the clock distributions: clock insertion delay, clock skew and clock jitter.

In the second part of the exercise we will also consider timing optimizations on the critical path of the design. The Placement/Timing exercise already covered how to obtain timing closure by specifying timing constraints. With this exercise we will go more in depth: we will understand how modifications to the RTL of our design can affect timing.

Preparation

Student Task 1

For ETH users:

- Setup the working directory for this exercise by calling our install script and navigating to the newly created directory. **Important**: make sure you install your exercises on the local hard disks rather than in the accounts home directories (which have a limited disk quota). For this sake, create a directory with your username on the local hard disk (e.g. `/scratch/vlsi2_XXfs26/`), and install the exercise files there. Check out the dedicated [Exercise setup]() page for more information.

```
sh > mkdir -p /scratch/$USER
sh > cd /scratch/$USER
sh > vlsi2 ex08
sh > cd ex08
```

Alternatively, you can directly call the install script for each exercise which you can find under `/home/vlsi2/ex08/install.sh`.

For non-ETH users:

Note: The exercises have been recently updated for the Spring semester 2026. It is possible that some exercise branches on Github require some additional time to be updated accordingly. Please reach out to a maintainer in case of issues.

- Retrieve the files from this exercise from Github:

```
sh > git clone https://github.com/pulp-platform/croc.git
sh > cd croc
sh > git checkout ex08
```

(A) How to design the clock tree?

The fan-out of a net refers to the number of inputs driven by a particular output. High fan-out nets (that drive hundreds or even thousands of inputs) may need to be handled differently from standard interconnections. Every synchronous circuit has at least one high fan-out net, namely the clock net. The main problem with high fan-out nets is the large load capacitance that needs to be driven. Each driven input adds its own input capacitance to the total load capacitance and in addition, the interconnection required to distribute the signal to all these inputs increases the load capacitance further.

Our goal is to make sure that the clock signal reaches all endpoints (called *leaf cells*) more or less at the same time as efficiently as possible. We define the following basic properties for a clock tree:

- **Transition time**: This is the time it takes to change the logic level of a node (e.g. 0 to 1). The more load an output has to drive, the more time is required to charge this load. CMOS drivers consume additional short circuit current during the transition, therefore long transition times are not very welcome. Furthermore, noise on signals with long transition times can result in glitching. Most libraries set an upper limit for the transition time (for the technology we are using this is 2.5 ns for typical libraries). A tree of buffers can be inserted to lower the transition time so that the total load is shared between the buffers. The lower the desired transition time, the more buffers are required.
- **Clock Skew**: The difference between insertion delays of different end-points. To minimize skew, a balanced buffer tree has to be built. Generally, the lower the desired skew the more buffers are required.
- **Insertion delay**: The time required for the signal to travel from the driver to the end points. In most cases, this delay is the result of us engineering a clock tree with a certain number of levels as a result of our constraints on transition time and skew. In some cases, we may wish to maintain a minimum insertion delay, as we might have already accounted for a specific minimum clock insertion delay when dividing our design into smaller more manageable blocks.

Our main focus today is on the clock. As discussed, the transition time is bounded for each technology library. Higher clock transition times can modify the setup of a cell, but we can control this by assuming the largest setup for static timing analysis. Power is a concern because of the short circuit current, but the problem can be solved by building a tree of clock buffer, i.e. the clock tree. The insertion delay should be solved at an earlier stage in the design, by partitioning the design in blocks.

The skew is an open point and it must be controlled by designing a balanced clock tree. Controlling the skew is important because it will affect the timing. In particular, a difference between the rising edge of sending and receiving flip-flops of a sequential design can make the window available for the combinational delay larger or shorter and thus affect timing closure. Let's reason on this with the following task.

Student Task 2

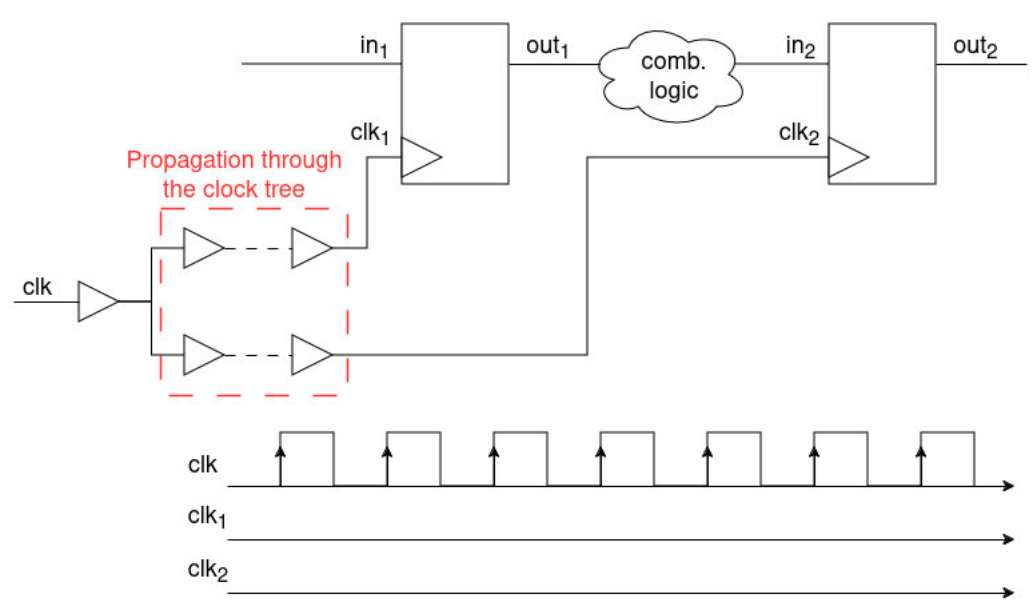

Figure 1: Simple register to register path.

Consider the simple register to register path in Figure 1.

- Mark the endpoints to measure the clock insertion delay and the clock skew on the figure.
- Do you think that the clock skew can influence the hold condition? Why?
- Do you think that the clock skew can influence the setup condition? Why?

Most designers would like to keep the clock skew under control, this also depends on your design. As an example, for a 10MHz clock a clock skew of 1ns is only 1% of the clock period and is probably acceptable. But for a 200MHz clock the same skew equals to 20% of the clock period. Reducing the clock skew will come at a price, you will insert buffers (area) that will have a high activity rate (power) and will change your I/O timing through the insertion delay. As in many cases, finding a reasonable balance between minimizing clock skew and adding overhead should be sought.

With the next task we will cover the basics of how to build a balanced clock tree. The outcome of this process should be similar to the tree described in figure 2.

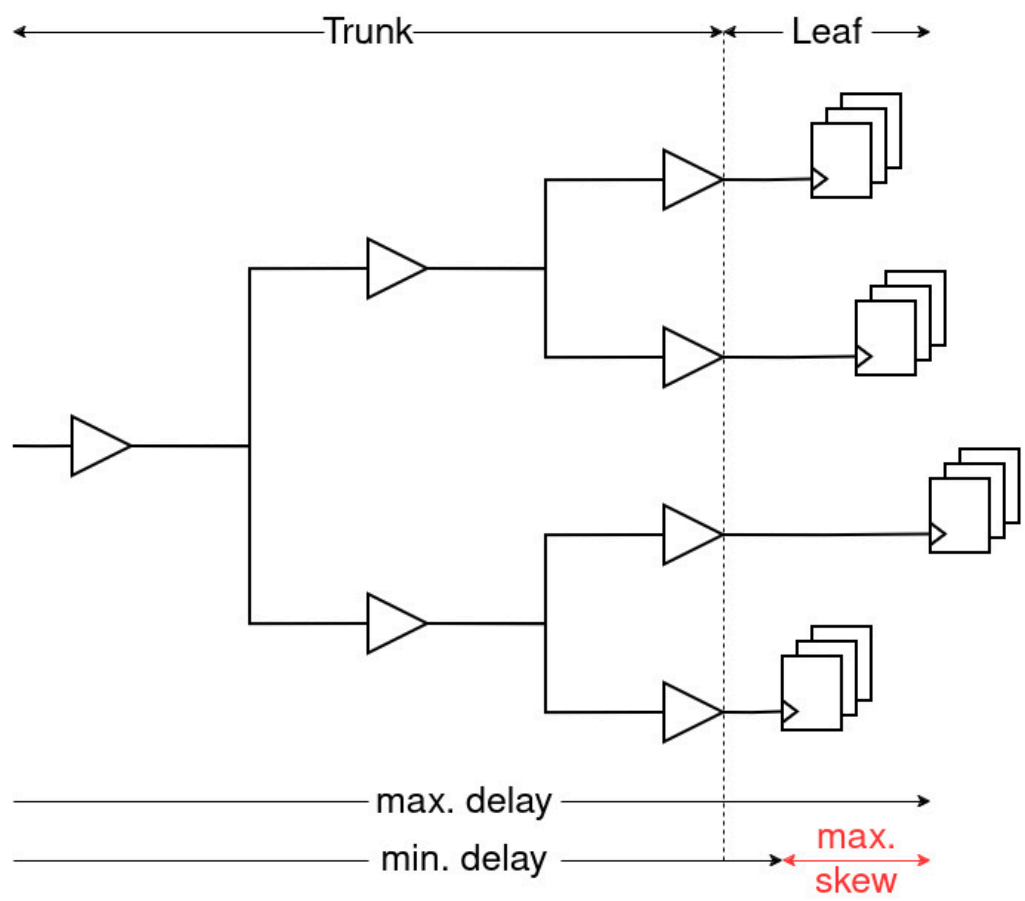

Figure 2: Clock tree.

Student Task 3

In this task, we will initialize our setup and load some pre-placed designs that are ready for clock-tree synthesis. You are encouraged to note down these steps in a `.tcl` script to save your progress and to speed up the next steps. Update the cockpit, then start Openroad. The GUI can be started by adding a `-gui` flag to the command to launch openroad or by typing `gui::show` in the Openroad shell.

```
sh > cd croc
sh > icdesign ihp13 -update all -nogui
sh > cd openroad
sh > oseda -2025.01 openroad
```

Set the project name and the name of the top design in the project:

```
OpenROAD > set proj_name croc
OpenROAD > set top_design croc_chip
```

Source the `reports.tcl` and the `checkpoint.tcl` scripts, containing helper functions to write reports and save checkpoints of the design. Source the technology information in `init_tech.tcl`. This includes the technology liberty (`.lib`) files.

```
OpenROAD > set report_dir ./reports
OpenROAD > source scripts/reports.tcl
OpenROAD > source scripts/checkpoint.tcl
OpenROAD > source scripts/init_tech.tcl
```

Load the netlist of a post-placement croc. Additionally, also include the Synopsys design constraint (`.sdc`) file:

```
OpenROAD > read_db ../../ex-files/out_dpl@80MHz/croc.dpl.odb
OpenROAD > read_sdc ../../ex-files/out_dpl@80MHz/croc.dpl.sdc
```

If you saved your progress in a `.tcl` script you can now repeat these startup operations by launching OpenRoad and sourcing the script. For example:

```
OpenROAD > oseda -2025.01 openroad scripts/YOUR_NAME.tcl
```

Open the GUI (`gui::show`). Select `Windows -> Clock Tree Viewer` and click `Update`. Does the clock distribution look like the image above?

Now, let's move to clock synthesis.

Student Task 4

First, we need some preparation. We must identify the clock nets and estimate the parasitics following the placement stage. We need to unset the don't touch status for the clock net. This was set before placement to avoid the optimization of the clock net, which is instead to be optimized and sized during clock tree synthesis. Finally, we will fix the inverters on the clock net by making sure that there are no inverters with fan-out > 1, to make the clock tree seen by clock tree synthesis simple from the start.

```
OpenROAD > set clock_nets [get_nets -of_objects [get_pins -of_objects "*_reg" -filter "name == CLK"]]
OpenROAD > set_wire_rc -clock -layer Metal4
OpenROAD > set_wire_rc -signal -layer Metal4
OpenROAD > estimate_parasitics -placement
OpenROAD > unset_dont_touch $clock_nets
OpenROAD > repair_clock_inverters
```

We can build our clock tree. The basic ingredients we need are the list of buffers that the tree will use and the root buffer, both specified in the `tech_init.tcl` file. We also give a basic flag to the command, which instructs it not to place clock buffers on top hard macros:

```
OpenROAD > clock_tree_synthesis -buf_list $ctsBuf -root_buf $ctsBufRoot -obstruction_aware
```

Open the GUI (`gui::show`). Explore the generated clock tree using the GUI widget (selecting `Windows -> Clock Tree Viewer` and clicking `Update`). Does the generated clock tree look like the image above?

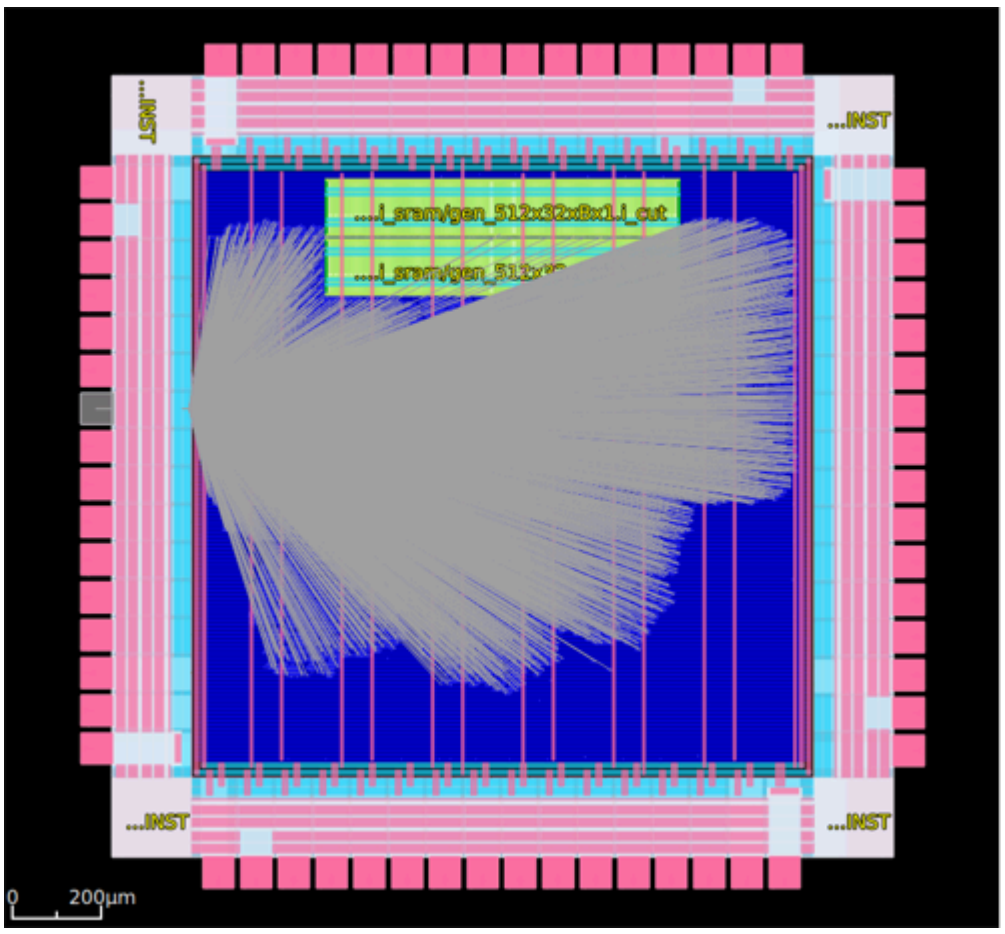

Figure 3: Before clock tree synthesis, the clock signal is directly connected from the clock pin of the chip to the flip-flops of the design.

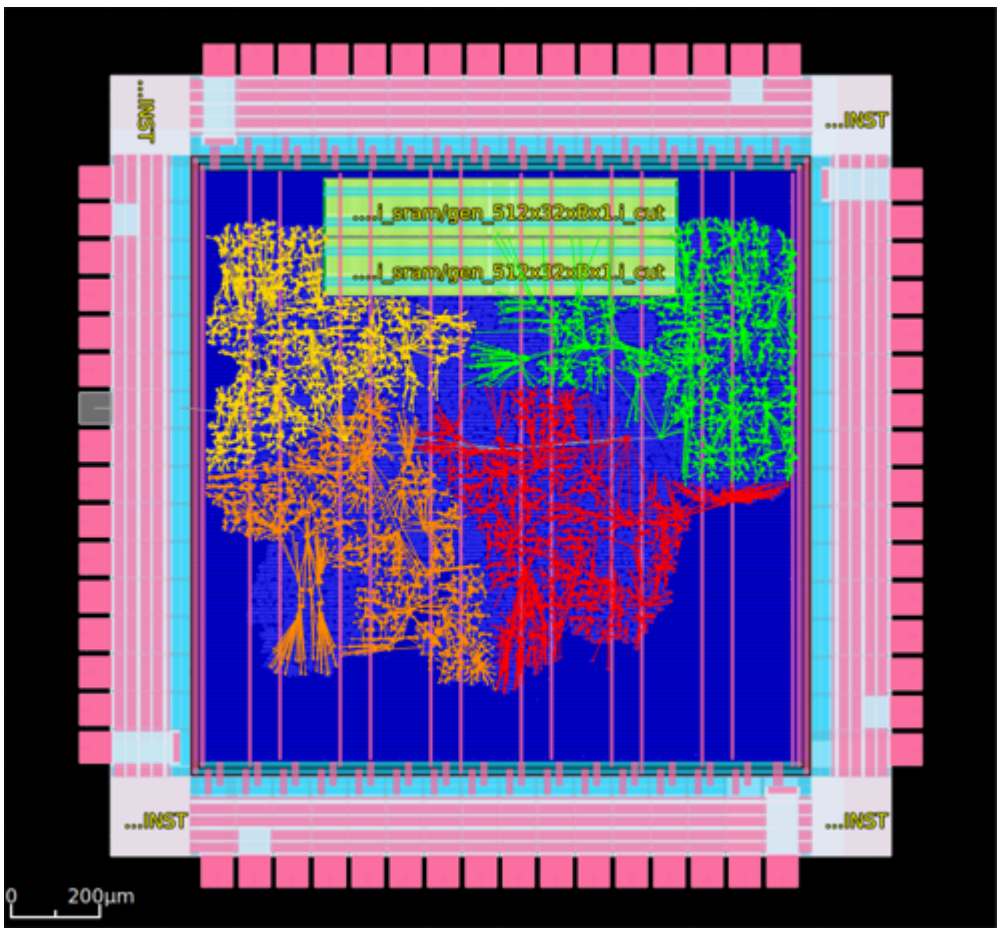

Figure 4: After building the clock tree, the clock signal is distributed according to the described tree structure. Clock buffers are added to regenerate the signal.

Especially if you have a bigger chip, the root buffer of the trunk might be far away from the clock input, this can be fixed using:

```
OpenROAD > repair_clock_nets
```

The following commands report information on the quality of the synthesised clock tree. In particular, they report the size of the tree in terms of the number of buffers and sinks, the max skew on both the falling edge and the rising edge of the clock.

```
OpenROAD > report_cts
OpenROAD > report_clock_latency -clock clk_sys
```

Please note down their output for the next tasks (we only care about the rising edge).

When building a clock tree we would like to distribute the sinks homogeneously in the design and to give it a balanced shape. This helps to create paths with the same distance from the clock input and to minimize the skew.

Student Task 5

There are two tricks to improve the quality of the synthesized clock tree: the first consists of defining clusters of sinks for clock distribution before building the H-shaped clock tree, and the second consists of balancing the logic levels in each level of the clock tree to have a more uniform distribution of the signal. Modify the tree synthesis command in your script as follows, and run your clock tree design again. How does the clock tree change using these solutions?

```
OpenROAD > clock_tree_synthesis -buf_list $ctsBuf -root_buf $ctsBufRoot -sink_clustering_enable -balance_levels -
obstruction_aware
```

Important metrics other than insertion delay and skew to consider when sizing the clock tree are the power burned in the clock network and the increase in the area of the design caused by the insertion of the clock tree.

Student Task 6

You can report the area of the logic cells used by the design with the `report_design_area` command. You can report an estimate of the power consumed by the clock by running the `report_power -corner tt` command in OpenROAD.

- What are the results for the clock tree that you synthesized? Please note down the results for later comparisons.

Consider the design `../../ex-files/out_dpl@125MHz/croc.dpl.odb`, which was placed with a different frequency target (125MHz compared to 80MHz). Load this design using your script and build the clock tree as discussed in the previous tasks.

- How much is the skew in percentage to the clock period?
- Try to reduce the skew; for example, you can launch your flow with an additional flag to the clock tree synthesis command and change the size of sink clusters (`-sink_clustering_max_diameter`, the default size is 50um). Are there changes from the point of view of the power and area consumed? Are other parameters also affected?

As a concluding remark on clock tree synthesis, we want to discuss the clock jitter. The clock jitter is a displacement of the clock edge, causing variations in the clock period. The jitter can be caused by an unstable clock source or by temperature and power supply variations, which make it relevant for large designs. The occurrence of jitter is unpredictable at design-time, we can therefore optimize the circuit on setup and hold, specifying an additional safety margin: the clock uncertainty. The uncertainty is a deterministic reduction of setup and hold margin that can be used to optimize the circuit for the worst-case scenario. We select an appropriate value in the `constraints.sdc` file.

(B) Post-CTS fixes

After building the clock tree there are some additional steps that you will have to go through. In particular, by synthesizing the clock tree, we added to the design clock buffers. This has two main effects. First, the propagation of the clock from the clock pin to the flip-flops of the design is no more ideal (zero-delay), but it has to take into account the propagation across the clock buffers. Second, the clock buffers have to be legalized in the placed design; that is, we need to fix the placement of the cells, to move the overlapping ones and to align with the row.

Student Task 7

Let's look at timing. Let's see first if the timing is violated.

```
OpenROAD > report_checks -path_group clk_sys
```

It seems there is no error. However, now we have a clock tree. Therefore, the timing analysis should take into account the propagation of the signal across the tree. Let's see what happens when we add the clock delays:

```
OpenROAD > estimate_parasitics -placement
OpenROAD > set_propagated_clock [all_clocks]
OpenROAD > report_checks -path_group clk_sys
```

What would you do now? This timing fix is what we need (add it to your script):

```
OpenROAD > repair_design -verbose
OpenROAD > repair_timing -setup -skip_pin_swap -verbose
```

We should now check the placement. Let's run a check on the placement of the design. Is everything okay? Are there some cells un-placed? Which ones?

```
OpenROAD > check_placement -verbose
```

It seems that we still need to place the buffers of the clock tree in our design and the timing optimization cells that where generated by the previous `repair_timing` command. Please add to your `.tcl` script:

```
OpenROAD > detailed_placement
OpenROAD > check_placement -verbose
```

(C) Other timing optimizations

With the next couple of tasks, we will consider how to fix timing violations and optimize the critical path of a design modifying the RTL, to address issues such as long combinational paths, unbalanced pipeline stages and high fan-out nets. Let's start by analyzing the timing report of a finished design.

Student Task 8

Start openroad with `oseda -2025.01 openroad` and load the netlist of a placed and routed croc, the technology liberty (`.lib`) files, and the Synopsys design constraint (`.sdc`) file, following the same steps of the previous part of the exercise. If you wrote down the required steps in a `.tcl` script you can reuse it to load the new design.

```
OpenROAD > read_db ../../ex-files/out_finished_unoptimized/croc.odb
OpenROAD > read_sdc ../../ex-files/out_finished_unoptimized/croc.sdc
```

The `.sdc` file defines the setup and hold timing constraints in different corners and sets the target frequency for all the clocks of the design. The `.lib` file is an ASCII representation of timing and power parameters associated with cells inside the ihp130 standard cell library. The `init_tech.tcl` script loads the `.lib` files for the given technology. The `.lib` file is a timing model file that contains cells delays, cells transition times, setup and hold time requirements of the technology cells. So the `.lib` file contains the timing and electrical characteristics of a cell or macros. The documentation files contain a summary of this information.

Student Task 9

Let's print the critical path for the different clocks of the design:

```
OpenROAD > set_wire_rc -clock -layer Metal4
OpenROAD > set_wire_rc -signal -layer Metal4
OpenROAD > estimate_parasitics -global_routing
OpenROAD > set_propagated_clock [all_clocks]
OpenROAD > report_checks -path_delay max -fields {slew cap input nets fanout} -format full_clock_expanded
```

- What is the longest path of the design for the clk_sys clock? Where is the startpoint? Where is the endpoint?
- Draw a diagram of the longest path with the main modules it crosses. Show the diagram to a teaching assistant.

Restart the tool and report now the timing for the `../../ex-files/out_finished` croc chip.

```
OpenROAD > read_db ../../ex-files/out_finished/croc.odb
OpenROAD > read_sdc ../../ex-files/out_finished/croc.sdc
OpenROAD > set_wire_rc -clock -layer Metal4
OpenROAD > set_wire_rc -signal -layer Metal4
OpenROAD > estimate_parasitics -global_routing
OpenROAD > set_propagated_clock [all_clocks]
OpenROAD > report_checks -path_delay max -fields {slew cap input nets fanout} -format full_clock_expanded
```

- What is the critical path for the `clk_sys`? Draw the main elements involved and show the picture to a teaching assistant.
- Is the result different from the timing of `./out_finished_unoptimized` chip?
- What changed between the two designs? Hint: the `finished` design was implemented using the exact same flow used for `finished_unoptimized` in `./scripts/chip.tcl`.

The architecture of your design specified in the RTL determines the timing of the final circuit beyond any possible optimization described until now. Potential sources of setup timing violations are: long combinational paths, unbalanced pipeline stages, and nets with high fan-in or fan-out.

Student Task 10

- Convince yourself that the RTL `./rtl` used to generate the unoptimized croc chip is valid, by running a functional simulation.
- Spot the differences between the RTL of the CVE2 core used for this exercise and the one in the GitHub repo of croc `https://github.com/pulp-platform/croc` and correct the unoptimized RTL (to identify the differences you can use diff on the `./rtl/cve2_core.sv`, `cve2_id_stage.sv`, and `cve2_if_stage.sv` files. Describe the changes applied to improve the timing (draw a circuit diagram and show it to a teaching assistant). How was the fan-out of the IF-stage flop-flops reduced? How was the ID-stage balanced?
- Verify your changes by running an RTL simulation on the modified RTL. Run the synthesis, placement, and routing flow to verify your timing results.

If your timing results correspond to the ones obtained for the `./out_finished` design you are at the end of this exercise!

However, before concluding let us express a final note. In the previous task, we balanced a pipeline stage by moving a module from the ID-stage to the IF-stage of the CVE2 core. For timing adjustments, it is also possible to leverage the retiming functionalities of EDA tools. Retiming will move registers at the boundary of pipeline stages to optimize the timing on the critical path of the design.

You are done with this exercise. Please discuss your experience with teaching assistants.

The VLSI pages are part of the open source VLSI design course offered by the Integrated Systems Laboratory (https://iis.ee.ethz.ch) of ETH Zürich (https://www.ethz.ch), by Luca Benini (https://ee.ethz.ch/the-department/people-a-z/person-detail.luca-benini.html) and Frank K. Gürkaynak (https://iis-people.ee.ethz.ch/~kgf/). See full list of contributors. Unless otherwise noted content available under Creative Commons BY (https://creativecommons.org/licenses/by/4.0/).